

Assignment Report on

Secure Network Application Development Using TCP

Submitted By	
Student Name:	Shahan Uz Zaman
Roll Number:	CS-BTC24-23
Course / Subject Name	Summer internship on Cyber Security and Secure App Development

1. Objective

The objective of this assignment is to enhance the existing GUI-based multi-client TCP chat application by adding practical security mechanisms. The application developed in the previous assignment was extended instead of creating a new application.

The major security features implemented are username and password authentication, secure password storage using SHA-256 hashing, user registration, duplicate-login prevention, input validation, failed-login protection, session management, logout, and secure logging. The application continues to support multi-client communication using TCP sockets.

The assignment focuses on practical application security rather than advanced cryptography.

2. Security Features Implemented

The following security features were implemented:

- **User Registration and Authentication:**

A Sign Up option was added to the client GUI. New users can create an account using a username and password. After successful registration, the same credentials can be used for login.
- **Secure Password Storage:**

Passwords are never stored as plain text. The server uses `hashlib.sha256()` to generate a hash before storing credentials in `users.csv`.
- **Duplicate Login Prevention:**

The server maintains a list of currently logged-in users. A user who is already logged in cannot create another active session with the same username.
- **Input Validation:**

The server validates usernames, passwords, messages, and commands. Empty or invalid usernames, empty passwords, oversized messages, and unsupported commands are rejected.
- **Failed Login Protection:**

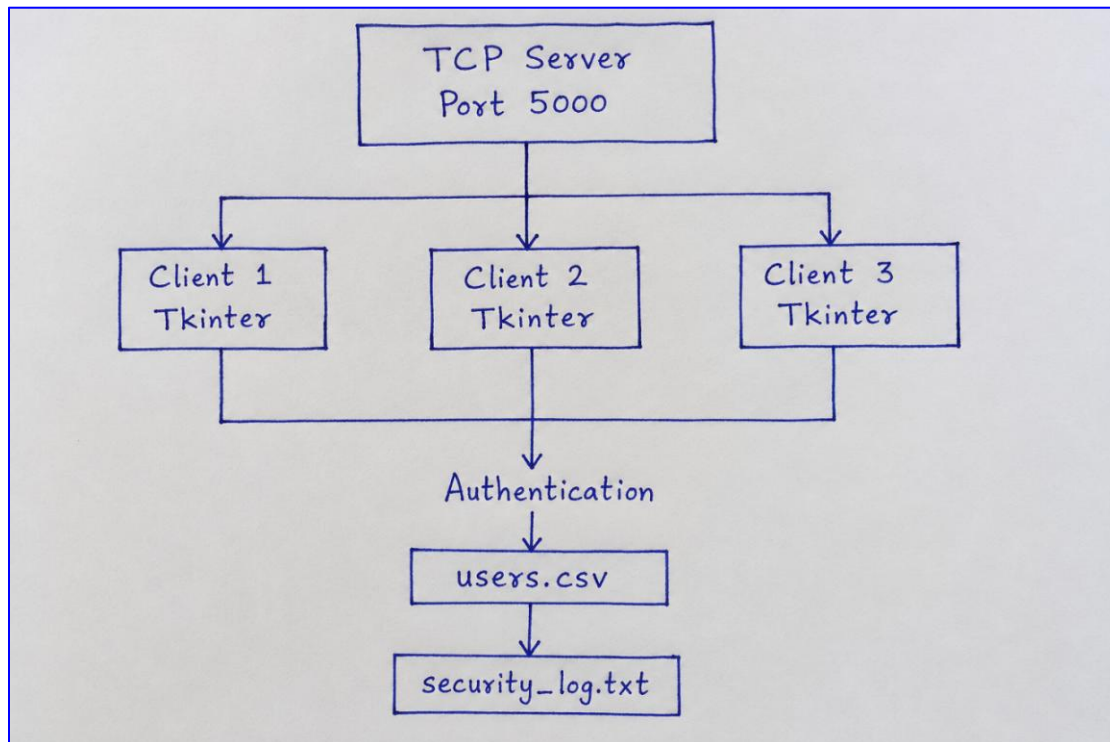
After five consecutive failed login attempts, the account is temporarily blocked to reduce repeated unauthorized login attempts.
- **Session Management:**

The application supports logout and inactivity timeout. Inactive sessions are removed and their network resources are released.
- **Secure Logging:**

Security events such as successful login, failed login, logout, and blocked attempts are recorded in `security_log.txt`. Passwords are never written to the log.

3. System Architecture

The application follows a client-server architecture.



The server manages multiple TCP clients using threads. It maintains online-user information and handles authentication, chat messages, private messages, commands, and session management.

The client provides a Tkinter-based graphical interface containing login/sign-up controls, chat display, online users, private messaging, and disconnect functionality.

The original multi-client architecture was reused as required by the assignment.

4. Implementation

The application consists mainly of:

- server.py
- client_gui.py
- users.csv
- security_log.txt

The server listens on TCP port 5000. When a client connects, it can register a new account or authenticate using an existing account.

For password storage, the password is converted into a SHA-256 hash:

Password
↓
SHA-256
↓
Password Hash
↓
users.csv

Only the hash is stored in the credential database.

The server also maintains active sessions in memory. This allows it to identify duplicate logins and remove disconnected or timed-out users.

The existing chat functions such as broadcast messaging, private messaging, online-user listing, and logout were retained and integrated with the security layer.

5. Testing

The application was tested using different authentication and communication scenarios.

Test Case	Expected Result
New user Sign Up	Account created
Existing username Sign Up	Registration rejected
Correct username/password	Login successful
Incorrect password	Login rejected
Five failed attempts	Temporary blocking
Duplicate login	Second session rejected
Empty username	Rejected
Empty password	Rejected
Oversized message	Rejected
Broadcast message	Delivered to clients
Private message	Delivered to target user
Logout	Session terminated
Inactivity timeout	Session removed

The testing demonstrates that the security mechanisms work together with the existing multi-client TCP communication system.

6. Wireshark Verification

Wireshark was used to observe TCP communication between the client and server. The required Wireshark filter was:

tcp.port == 5000

The following scenarios were captured:

- **Successful Login:**
A valid username and password were submitted and the corresponding TCP communication was observed.
- **Failed Login:**
An incorrect password was submitted and the failed authentication communication was captured.
- **Authenticated Communication:**
After successful authentication, normal chat messages were exchanged between clients and the server. The TCP packets were captured using the same filter.

- **Logout:**

The client disconnected from the server and the corresponding TCP communication was captured.

Additional screenshots were taken for user registration and failed-login protection.

The assignment specifically requires capturing login, failed login, authenticated communication, and logout using TCP port 5000.

7. Conclusion

This assignment successfully enhanced the previous multi-client TCP chat application with practical security features. User authentication, registration, SHA-256 password storage, duplicate-login prevention, input validation, failed-login protection, session timeout, logout, and secure logging were implemented.

Testing confirmed that valid users can authenticate and communicate while invalid authentication attempts and duplicate sessions are controlled. Wireshark was also used to verify TCP communication during login, failed login, authenticated communication, and logout.

The project demonstrates how security mechanisms can be integrated into an existing network application without completely redesigning the original application. The final implementation provides a more secure and reliable multi-client TCP chat system suitable for the assignment requirements.